# Programs Reach What Statistics Cannot, but Search Is Not Yet Paid For: A Bit-Priced Program Library on Synthetic and Real Streams

Eric Lam (independent researcher, Toronto)

## Abstract

We build a byte-stream machine that prices every byte in bits, searches for small programs when a field stays expensive, keeps a program only while it saves more bits than a fixed charge, and evicts it when it stops paying. We test four claims against controls on synthetic streams and on real files the machine did not write. Programs reach computational structure that gzip and xz do not model: on a never-repeating counter stream, the machine reaches 0.459 to 0.492 bits per byte against 1.05 to 1.16 for xz, and against 2.09 to 2.17 for the same coder without programs. On repetitive streams the program layer contributes nothing, and xz wins. Reusing library entries lowers the search cost per new rule in a 20-entry staircase, but the benefit is bounded. Abstracting the library shrinks it by 33% without changing predictions, and a learned schema does not reduce search. A drive that spends effort where bits fall fastest loses to uniform reading on held-out bits in 3 of 3 seeds. On real logs the machine does not beat xz. We report every result, including the failures, and the one tripwire test that was not passed.

## 1 Introduction

Learning as compression has a long theory: Solomonoff's induction, Rissanen's minimum description length (MDL), and Hutter's statement of compression as intelligence. We do not claim that theory. The question here is narrower. If a machine pays for every byte in bits using only what it has already seen, and may add a program to its description when that lowers total bits, does the program layer reach structure that common compressors miss, and does the machine get cheaper to run as its library grows?

The work is a phase of a longer program (folder `Javis`). Earlier phases tried a neural sequence learner on generated rule streams and did not learn never-seen rules from in-context examples at the scales tested (137k to 10M parameters). This paper covers the symbolic phase that followed. It contains no neural network. Its design draws on DreamCoder, which library-learns programs from task solutions, and on Stitch, a faster abstraction learner (references 1–2).

Our contributions are measurements, not a new theory: (i) a controlled test of complementarity between programs and statistics, including on data we did not write; (ii) a test of compounding, where reuse should make later structure cheaper; (iii) a negative test of abstraction as a search prior; (iv) a negative test of a progress-chasing drive; and (v) an honest record of a tripwire that was not passed.

## 2 Method

**Coder.** Each byte costs `-log2 p` bits, where `p` comes from a blend of an order-k count model and a set of programs. The count model adapts on every byte.

**Programs.** A program predicts one numeric field of a line from earlier fields in the same line or from earlier lines. The forms are `const`, `delta` (a field plus an inferred constant), `copy`, and `op` (a binary operation on two sources). Sources may be earlier library entries, so a new rule can be written in terms of an old one. A program is kept only if it has already saved more bits than a flat charge `PROGRAM_BITS = 40` for naming it. Programs are evicted when they stop paying.

**Controls.** Every result is compared with (a) the same coder with search disabled, and (b) gzip and xz on the same bytes. Where structure is supposed to be absent, a scrambled or shuffled arm asserts that the machine should not win. Bits are the only score. No output is hand-graded.

**Data.** Synthetic streams of about 6 KB to 9 KB (exp25, exp26, exp27, exp28); a 143 KB English book (exp25, exp31); the first 1,000,000 bytes of four Windows and ComfyUI logs (exp32); the Our World in Data COVID series, about 990 KB (exp33, two column orders); and a 63 KB JHU CSSE daily report (exp33). Real files were not generated by us.

**Runs.** Most experiments use three seeds. The project contract required bars to be fixed before each run; the bar outcomes are recorded in each run's `result.json`.

## 3 Experiments and Results

**Table 1. Complementarity and compounding (exp25–exp28).**

| Test | Machine | Control | Compressors | Verdict |
|---|---|---|---|---|
| Hidden rules, synthetic | 2.353 bpb, hit 1.000 | 2.923 bpb | — | PASS |
| Hidden rules, answers randomised | 0 programs | identical to control | — | PASS (must-not-win) |
| Computational, never repeats (3 seeds) | 0.459–0.492 bpb | 2.09–2.17 | gzip 2.35–2.51; xz 1.05–1.16 | PASS (C1) |
| Repetitive, 20 lines cycled (3 seeds) | 0.386–0.389 bpb, 0 programs | 0.386–0.389 | gzip 0.366–0.370; xz 0.545–0.547 | part of C1 |
| Staircase, reuse vs none | 2.441 bpb, 6 programs (5 built on earlier ones) | 2.871 bpb without reuse | — | PASS (C2a) |
| Six unrelated families | 1.868 bpb | 1.868 bpb plain | — | PASS (C2c) |
| Rule changes mid-stream, eviction | 2.982 bpb | 3.415 bpb with eviction off | — | PASS (E1) |
| 20-family staircase, search cost | 197–264 → 3.2–5.2 candidates per rule | no reuse: 5 programs | — | PASS, bounded to 20 entries |

**Table 2. Abstraction, transfer and symbols (exp29–exp31).**

| Test | Result | Control | Verdict |
|---|---|---|---|
| Library abstraction, 20-family staircase | library 1280 → 855 bits (−33%), 15 of 20 rules covered; 2.671 → 2.623 bpb | predictions unchanged | PASS |
| Learned schema, cold vs warm, candidates per rule | cold 90.2–135.9; warm 101.8–138.5 | wrong schema 389.6–438.0 | FAIL (E30a) |
| Shared word between fields | words as objects 0.234–0.237 bpb, 3 programs | independent-field control 0.285–0.286, 2 programs | PASS |
| Real English book, symbols on vs off | 2.177 bpb both | — | no gain on prose |

**Table 3. Real files (exp32, exp33). Bits per byte.**

| File | Machine (objects for exp32 rows; background for OWID and JHU rows) | No-search control | gzip | xz | Verdict |
|---|---|---|---|---|---|
| CBS.log | 0.214 | 0.243 | 0.201 | 0.136 | tripwire FAIL |
| dism.log | 0.187 | 0.200 | 0.258 | 0.141 | tripwire FAIL |
| storgp | 0.440 | 0.456 | 0.278 | 0.192 | tripwire FAIL |
| ComfyUI | 0.227 | 0.252 | 0.294 | 0.172 | tripwire FAIL |
| OWID, columns reordered (words as background) | 0.614 | 0.858 | 0.871 | 0.433 | R1–R4 PASS; loses to xz |
| OWID as shipped | 1.109 | 1.109 | 0.897 | 0.451 | accumulator not found (predicted) |
| JHU, words as background | 2.606 | 2.630 | 3.031 | 2.632 | pre-registered null FAILED |

The exp32 result file records both bars as true; the honest verdict, written in the project's ledger, is FAIL, because two of the admitted "findings" reduce to laundering channels (indirection through the library and constant fields). A must-not-win arm that scrambles only near-constant fields cannot catch this.

**Table 4. Drive (exp34). Held-out bits on four streams, budget 204,800 bytes.**

| Scheduler | Held-out bits, seeds 0/1/2 | Share of budget on noise |
|---|---|---|
| Uniform (control) | 142,298 / 142,741 / 142,511 | 25% |
| Progress (bits falling fastest) | 149,268 / 149,584 / 149,318 | 49% |
| Level (bits highest) | 150,362 / 150,678 / 150,412 | 49% |

All three drive bars fail in 3 of 3 seeds. Per-chunk progress shows why. Useful streams drop to a plateau quickly. A noise stream keeps improving slowly, so a progress-chaser keeps returning to it.

## 4 Discussion

The complementarity result is the cleanest positive finding. Programs and statistics fail on different kinds of structure, and each is silent on the other's. Where a program layer adds nothing (repetitive data), it adds nothing exactly: the machine's bits equal the control's. That is a useful sanity property for a must-not-win arm, and it is why the complementarity claim is credible.

Compounding holds only inside the envelope tested. The 20-family staircase used a small form set (`const`, `delta`, one operation). When we tried to force a newer parent, the test degenerated, so the result shows that reuse lowers search cost in this space, not that it always does. A richer program space is needed to test whether search cost ever explodes.

The negative results are as informative as the positive ones. A learned schema for candidate order did not help (E30a). A wrong schema cost 6.7 times more search, so candidate order does matter, but the abstraction we built generalised away the part that would have helped. The drive failed because compression progress rewards slow, permanent improvement on noise over fast, saturating improvement on useful streams. An earlier explanation based on estimator variance was checked and rejected.

On real logs the machine loses to xz on total bits. These logs are repetitive, which is the model class xz fits. Structure that programs can express (the OWID accumulator, reached after a column reorder) is real, but it is a small share of the bits in those files. The JHU identity that was pre-registered as unreachable was built anyway, by composing programs. That is a reason to treat pre-registered nulls as predictions that can fail, not as proof.

## 5 Limitations

- Synthetic streams are small (about 6 KB to 9 KB). The real files are capped at 1 MB, except JHU and OWID.
- The program forms are limited to `const`, `delta`, `copy` and a two-operand `op`. The search budget (1000 candidates) is a constraint. Unbudgeted search changed bits on only one of eight file and representation arms at 1 MB, and on dism it cost 48 times the wall-clock time.
- Bits exclude any model cost. This phase contains no neural model. A transformer prior would cost 27,336,704 bits (854,272 parameters at 32 bits) under the same accounting, which we have not measured.
- Real baselines are gzip and xz only. We did not compare against a neural compressor on the same files.
- Some arms are single-seed, and the tripwire was run once per file.
- On OWID the scrambled control was live (it changed 76.5–76.8% of field values and admitted zero programs). On the tripwire files, a scramble redraws near-constant fields to themselves, so the must-not-win arm was vacuous for those fields.

## 6 Conclusion

A bit-priced program library reaches computational structure that gzip and xz cannot model, and it does nothing on repetitive data where they win. Reuse lowers the search cost of new rules in a bounded

envelope. Three claims did not hold: a learned abstraction did not speed search, a progress-chasing drive lost to uniform reading, and the machine did not beat xz on real logs. The tripwire test remains open. The next step, planned but not run, is a frozen transformer prior combined with the library, with its cost stated in bits.

## References


1. Ellis et al., DreamCoder. arXiv:2006.08381 (Ellis et al., 2020; PLDI 2021; Phil. Trans. R. Soc. A 2023).
2. Stitch. POPL 2023. arXiv:2211.16605.
3. Schmidhuber, J. Formal Theory of Creativity, Fun and Intrinsic Motivation. IEEE TAMD, 2010 (the project notes date the underlying work 2008/2009).
4. Grünwald, P. A tutorial on MDL. arXiv:math/0406077.
5. Bellard, F. NNCP. https://bellard.org/nncp/
6. gzip and xz (baseline compressors). Versions not recorded in the project notes.

Also named in the project notes without further detail (not cited in the text): Solomonoff's induction; Rissanen's MDL; Hutter's compression-as-intelligence statement.